TD : tester les performances de l'API avec k6

Objectif

Vous allez utiliser k6 pour envoyer des requêtes à notre API Laravel, observer ses réponses et comprendre ce qui se passe lorsque plusieurs utilisateurs virtuels demandent la liste des documents. Vous comparerez aussi une lecture avec et sans cache, puis vous observerez la réponse HTTP 429 produite par la limitation du jour 4.

k6 est un outil distinct de Laravel. Nous l'exécuterons dans Docker : aucun paquet Composer supplémentaire n'est nécessaire. Le test utilise l'API locale et un compte fictif. Les requêtes du scénario lisent les documents, sans en créer ni en supprimer.

Si vous consultez ce guide sur la branche de correction : le scénario et l'interrupteur de cache sont déjà présents. Les étapes ci-dessous expliquent comment les construire depuis la branche de départ ; pour exécuter le script fourni, consultez aussi « Utiliser la correction » en fin de guide.

Avant de commencer

Ouvrez dans VS Code le dossier **Laravel** qui contient `artisan` et `compose.yaml`. Dans **Terminal > Nouveau terminal**, choisissez PowerShell. Toutes les commandes de ce guide sont à lancer dans ce terminal, à la racine du projet.

- Partez de votre travail terminé du jour 4 ou de la branche **`jour4-rate-limiting-documents`**. La branche **`jour5-k6-performance-documents`** contient la correction.
- Docker Desktop doit être démarré. Laravel, PostgreSQL, Valkey et les comptes fictifs du jour 3 doivent déjà être configurés.
- Utilisez uniquement votre base locale de formation. Ne recréez pas `.env` ou `APP_KEY` et ne supprimez pas les volumes Docker.
- Sauvegardez votre travail avant de changer de branche. Si Git refuse le changement, ne le forcez pas ; demandez au formateur comment conserver vos fichiers.

```
git branch --show-current
git status
docker compose ps
```

À vérifier : la branche attendue est affichée et les services `app`, `db` et `valkey` sont démarrés. Si besoin, démarrez les services du projet avec `docker compose up -d`. Si le projet n'a pas encore été configuré, reprenez les guides de démarrage et de Valkey avant de continuer.

Partie 1 : découvrir k6

1. Vérifier Docker et k6

Dans le terminal du projet, lancez :

```
docker compose exec valkey valkey-cli ping
docker compose exec app php artisan config:show cache.default
docker run --rm grafana/k6:1.4.2 version
```

À vérifier : Valkey répond **PONG**, Laravel affiche `redis` comme cache par défaut, et la dernière commande affiche une version de k6. Lors de la première utilisation, Docker peut télécharger l'image officielle : une connexion Internet est alors nécessaire.

`docker run` démarre ici un conteneur temporaire, `--rm` le supprime après l'exécution et `version` demande à k6 d'afficher sa version. Votre application Laravel et ses données restent dans leurs propres conteneurs.

2. Comprendre le scénario

Un **utilisateur virtuel**, ou **VU**, est un utilisateur simulé par k6. Il exécute les instructions d'un scénario : par exemple, envoyer une requête pour lire les documents. Nous choisissons le nombre de VU et la durée du test. Un VU répète le scénario pendant cette durée.

Notre route `GET /api/documents` attend un token Sanctum : elle ne renvoie que les documents de l'utilisateur connecté. Dans ce TD, les VU utilisent le même token. Laravel les reconnaît donc comme **un seul compte**, même si k6 simule plusieurs clients.

Les codes de réponse à reconnaître :

Code	Ce qu'il indique ici
`200`	La liste a été lue avec succès. Elle peut être vide.
`401`	L'utilisateur n'a pas été authentifié : le token manque, a expiré ou n'est pas valide.
`429`	Le compte a dépassé la limite de requêtes sur la liste.

À vous : si deux VU utilisent le token d'Amina, Laravel voit-il deux comptes différents ? Le nombre de VU est-il forcément le nombre de requêtes par seconde ?

Partie 2 : créer un premier scénario

3. Obtenir un token de formation

Le compte `amina@example.test` a été préparé au jour 3. Dans le terminal PowerShell du projet, connectez-le :

```
$body = @{ email = 'amina@example.test'; password = 'Formation123!' } | ConvertTo-Json
$login = Invoke-RestMethod -Method Post -Uri 'http://localhost:8088/api/login' -ContentType 'application/json' -Body
$body
$env:K6_TOKEN = $login.token
```

La première ligne prépare les données JSON. La deuxième appelle la route de connexion. La troisième place le token reçu dans une **variable d'environnement de ce terminal** pour que le conteneur k6 puisse l'utiliser.

À vérifier : aucune erreur ne s'affiche. Pour vérifier la présence du token sans afficher sa valeur :

```
if ($env:K6_TOKEN) { 'Token reçu' } else { 'Token absent' }
```

Le token donne accès au compte : ne l'écrivez pas dans le script, ne le publiez pas sur Git et ne l'affichez pas dans une capture. La variable disparaît à la fermeture de ce terminal ; dans un nouveau terminal, refaites la connexion. Si le compte existait déjà avant le seeder du jour 3, son mot de passe n'a pas été modifié automatiquement : demandez les identifiants au formateur en cas d'échec.

4. Créer le fichier du scénario

Dans l'explorateur de VS Code, créez un dossier **`k6`** à la racine du projet. Dans ce dossier, créez un fichier **`documents.js`**. S'il existe déjà, vérifiez votre branche et votre travail avant de le remplacer.

Copiez le code suivant dans `k6/documents.js`, puis enregistrez :

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 1,
  duration: '3s',
};

export default function () {
  const response = http.get('http://app:8080/api/documents', {
    headers: {
      Authorization: `Bearer ${__ENV.K6_TOKEN}`,
      Accept: 'application/json',
    },
  });

  check(response, {
    'lecture réussie (200)': (r) => r.status === 200,
  });

  sleep(1);
}
```

Lisez chaque partie :

- `http` permet d'envoyer des requêtes HTTP. `check` sert à vérifier une réponse et `sleep` ajoute une pause.
- `vus: 1` demande un utilisateur virtuel et `duration: '3s'` fixe la durée du test à trois secondes.
- La fonction `default` décrit ce que chaque VU répète. Ici, elle lit la liste avec `http.get`.
- `Authorization: Bearer ...` transmet le token. `__ENV.K6_TOKEN` lit la variable fournie à k6, sans inscrire le token dans le fichier.
- Le check demande si le code vaut `200`. `sleep(1)` fait attendre chaque VU une seconde avant l'itération suivante.

`http://app:8080` est l'adresse du service Laravel **depuis le réseau Docker**. Depuis votre navigateur Windows, l'adresse reste `http://localhost:8088`.

5. Lancer le premier test

Dans le même terminal PowerShell, à la racine du projet :

```
docker run --rm --network formation-laravel_default -v "${PWD}:/scripts" -e K6_TOKEN grafana/k6:1.4.2 run
/scripts/k6/documents.js
```

`--network` connecte k6 au réseau Docker du projet ; `formation-laravel` est le nom défini dans `compose.yaml`. `-v` rend le dossier du projet accessible dans le conteneur à l'emplacement `/scripts`. `-e K6_TOKEN` lui transmet la variable du terminal. Enfin, `run` exécute notre fichier.

À vérifier : le terminal affiche un scénario avec **1 VU**, puis un résumé à la fin. Le check `lecture réussie (200)` doit réussir si le token est valide et si le quota du compte n'a pas déjà été utilisé. Une réponse de liste vide peut être correcte. Après un essai précédent, attendez au moins une minute sans lire la liste avec ce compte avant de recommencer.

À vous : où le token est-il stocké pendant le test ? Que changeriez-vous pour que chaque VU attende deux secondes ?

Partie 3 : comprendre les résultats

6. Lire le résumé du terminal

Dans le résultat k6, repérez les lignes suivantes. Leur présentation peut varier légèrement selon la version.

Mesure	Comment la lire
`checks_succeeded` et `checks_failed`	Part des vérifications réussies ou ratées. Ici, notre check attend le statut 200.
`http_req_duration`	Durée des requêtes : `avg` est la moyenne ; `p(95)` est une valeur sous laquelle se situent 95 % des mesures.
`http_reqs`	Nombre total de requêtes et débit moyen en requêtes par seconde.
`http_req_failed`	Part des requêtes que k6 considère en échec HTTP. Par défaut, les réponses 429 en font partie.
`iterations`	Nombre d'exécutions complètes du scénario.

Par exemple, si le résumé indique `http_reqs: 3`, trois requêtes ont été envoyées. Ne confondez pas le **nombre total de requêtes** avec le **nombre d'utilisateurs virtuels** : un même VU peut répéter son scénario plusieurs fois.

Un check raté s'affiche dans le résumé. Un check seul ne rend pas automatiquement le processus k6 en échec : pour définir une condition de réussite globale et un code de sortie d'échec, k6 propose les `thresholds`, hors de ce premier scénario.

À vous : notez le nombre de requêtes, leur durée moyenne et le résultat du check. Une réponse 429 a-t-elle effectué la même lecture qu'une réponse 200 ?

7. Observer le tableau de bord dans le navigateur

Dans `k6/documents.js`, remplacez temporairement `duration: '3s'` par `duration: '20s'` pour avoir le temps d'ouvrir le navigateur. Enregistrez, puis lancez :

```
docker run --rm --network formation-laravel_default -p 127.0.0.1:5665:5665 -v "${PWD}:/scripts" -e K6_TOKEN -e K6_WEB_DASHBOARD=true -e K6_WEB_DASHBOARD_HOST=0.0.0.0 -e K6_WEB_DASHBOARD_PERIOD=1s grafana/k6:1.4.2 run /scripts/k6/documents.js
```

Pendant le test, ouvrez **http://localhost:5665**. Le tableau de bord montre les mesures en cours sous forme de graphiques. Il sert à observer le test ; la commande continue à le lancer. Le tableau de bord s'arrête à la fin du test.

À vérifier : la page s'ouvre et les mesures évoluent. Avec un seul compte et un test de cette durée, plusieurs réponses `429` sont attendues après les cinq lectures autorisées. Remettez ensuite `duration: '3s'` dans votre fichier et enregistrez.

Partie 4 : faire varier la charge

8. Essayer plusieurs utilisateurs virtuels

Pour comparer les essais, complétez ce tableau avec vos propres résultats. Attendez **au moins 60 secondes sans lecture de la liste avec Amina** avant chaque essai : le compteur du compte doit expirer.

Essai	Valeur de `vus`	Durée	Requêtes	Check 200	Réponses 429 observées
A	1	3 secondes	À relever	À relever	À relever
B	2	3 secondes	À relever	À relever	À relever
C	5	3 secondes	À relever	À relever	À relever

1. Vérifiez `vus: 1` et `duration: '3s'`, puis relancez la commande de l'étape 5. Relevez les résultats.
2. Après expiration du quota, remplacez `vus: 1` par `vus: 2`, enregistrez et relancez.
3. Attendez de nouveau, remplacez `vus: 2` par `vus: 5`, puis relancez et complétez le tableau.

Tous les VU utilisent le même token. La limite de **cinq lectures par minute** leur est donc commune. Lorsque la charge augmente, les réponses 429 peuvent devenir nombreuses : cela montre le fonctionnement de la protection. L'affichage du check et `http_req_failed` signalent ces réponses comme des échecs de lecture, même si elles sont attendues pendant cette expérience.

À vous : pourquoi cinq VU ne peuvent-ils pas chacun effectuer cinq lectures avec le même token ? Pourquoi le nombre de requêtes par seconde n'est-il pas forcément égal au nombre de VU ?

Partie 5 : comparer la lecture sans cache et avec cache

9. Ajouter un interrupteur pour l'exercice

Nous voulons désactiver **uniquement le cache de la liste des documents** pour comparer les lectures. Valkey doit rester le cache général de Laravel afin de conserver notamment les compteurs de limitation.

Ouvrez `config/cache.php`. Repérez cette ligne :

```
    'default' => env('CACHE_STORE', 'database'),
```

Ajoutez juste après :

```
    'documents_enabled' => env('DOCUMENTS_CACHE_ENABLED', true),
```

Enregistrez. Cette option lit `DOCUMENTS_CACHE_ENABLED` depuis `.env`. Si la variable est absente, elle vaut `true` : la liste reste mise en cache par défaut.

Ouvrez ensuite `app/Http/Controllers/DocumentController.php`. Dans la méthode `index`, ajoutez les quatre lignes suivantes **avant** la ligne `$key = 'documents:user:...` :

```
        if (! config('cache.documents_enabled')) {
            return response()->json($request->user()->documents()->get());
        }
```

Conservez tout le code de cache existant après ce bloc. Si l'option est désactivée, Laravel lit directement les documents de l'utilisateur dans PostgreSQL et renvoie le résultat. Sinon, il continue vers le `Cache::remember` du jour 4.

Pour documenter ce nouveau réglage pour les autres étudiants, ajoutez aussi cette ligne dans `.env.example`, après `CACHE_STORE=redis` :

```
DOCUMENTS_CACHE_ENABLED=true
```

Ne copiez pas le contenu de votre `.env` personnel dans `.env.example`.

10. Mesurer la lecture sans cache

Dans votre fichier **`.env` existant**, ajoutez ou modifiez seulement :

```
DOCUMENTS_CACHE_ENABLED=false
```

Conservez `CACHE_STORE=redis`. Enregistrez, puis demandez à Laravel de retirer sa configuration compilée :

```
docker compose exec app php artisan config:clear
```

```
docker compose exec app php artisan config:show cache.documents_enabled
```

À vérifier : la seconde commande indique que l'option est désactivée. L'application utilise toujours Valkey pour son cache général et le limiteur.

Dans `k6/documents.js`, remettez `vus: 1` et `duration: '3s'`. Après expiration du quota, lancez la commande de l'étape 5. Vérifiez que toutes les réponses du check sont `200`, puis relevez le nombre de requêtes et `http_req_duration`.

11. Mesurer la lecture avec cache

Dans le même fichier `.env`, remplacez uniquement :

```
DOCUMENTS_CACHE_ENABLED=true
```

Enregistrez et relancez :

```
docker compose exec app php artisan config:clear
docker compose exec app php artisan config:show cache.documents_enabled
```

À vérifier : l'option est activée. Après expiration du quota, lancez une première fois le scénario : la première lecture remplit le cache s'il est vide. Attendez ensuite l'expiration du quota et relancez **le même scénario**. Notez les mesures des réponses réussies.

Situation	VU	Durée	Nombre de réponses 200	Durée moyenne
Sans cache de la liste	1	3 secondes	À relever	À relever
Avec cache de la liste déjà rempli	1	3 secondes	À relever	À relever

Pour une comparaison lisible, utilisez le même compte, les mêmes documents et les mêmes réglages. Si des `429` apparaissent, attendez l'expiration du quota et recommencez : comparer une réponse `429` à une lecture `200` serait trompeur. Le groupe `expected_response:true` de `http_req_duration` distingue les réponses attendues ; dans cet exercice, vérifiez aussi que le check indique uniquement des `200`.

À vérifier : dans les deux cas, la liste renvoyée reste celle du même utilisateur. Le résultat chronométré peut varier selon le poste et le nombre de documents. Trois lectures ne suffisent pas à prouver un gain : l'objectif est de comprendre et d'observer le mécanisme.

À vous : pourquoi n'avons-nous pas mis `CACHE_STORE=null` ? Que fait la première lecture lorsque la clé de cache n'existe pas ?

Partie 6 : observer HTTP 429

12. Envoyer six lectures successives

Dans `k6/documents.js`, remplacez temporairement le bloc `options` par :

```
export const options = {
  vus: 1,
  iterations: 6,
};
```

Dans la fonction, juste après le bloc `check(...)`, ajoutez cette ligne :

```
  console.log('Réponse HTTP : ' + response.status);
```

Retirez temporairement `sleep(1);` pour enchaîner les six requêtes. Enregistrez. Après au moins 60 secondes sans lecture de la liste avec Amina, lancez la commande de l'étape 5.

À vérifier : si aucun autre client n'a utilisé le quota d'Amina, le terminal affiche cinq réponses **200**, puis une réponse **429**. Le check 200 échoue sur la sixième réponse et `http_req_failed` la compte comme un échec HTTP : c'est attendu ici. Si le compte a déjà consommé une partie de son quota, le 429 peut arriver plus tôt.

Attendez de nouveau l'expiration du compteur avant un autre essai. Rétablissez ensuite `vus: 1`, `duration: '3s'` et `sleep(1);` si vous souhaitez reprendre le scénario initial.

À vous : quelle route est limitée ? Pourquoi les réponses 429 de cet exercice indiquent-elles que la protection fonctionne ?

Vérifications et difficultés

Situation	Vérification
Docker ne trouve pas l'image k6	Vérifiez que Docker Desktop est démarré et que le premier téléchargement dispose d'une connexion Internet.
`network formation-laravel_default not found`	Lancez `docker compose up -d` à la racine du projet ; vérifiez le nom du projet et son réseau si votre configuration diffère.
`Token absent` ou réponse `401`	Refaites la connexion dans le même terminal. Vérifiez le compte et le mot de passe de formation sans afficher le token.
`429` dès le début	Attendez au moins 60 secondes sans lecture avec le même compte ; un navigateur ou Postman peut aussi consommer son quota.
Réponse `429` à la connexion	La route de connexion possède sa propre limite. Attendez avant de réessayer.
Impossible de joindre `app:8080`	Vérifiez que `app` est démarré et que k6 utilise le réseau Docker du projet. Depuis Windows, utilisez `localhost:8088`.
Tableau de bord inaccessible	Ouvrez `localhost:5665` pendant le test, vérifiez l'option `-p` et que le port est libre.
L'option de cache ne change rien	Vérifiez le nom exact de la variable, le code du contrôleur, `config:clear` et la valeur de `config:show cache.documents_enabled`.
Comparaison cache étrange	Vérifiez que les réponses sont `200`, que les données et la charge sont identiques et que le cache a été rempli avant l'essai à chaud.

Comparer avec la correction

La branche `jour5-k6-performance-documents` contient une version paramétrable du scénario et l'interrupteur de cache. La correction ne sera pas forcément identique ligne par ligne à votre premier script : comparez d'abord ce que fait chaque partie.

Dans votre dépôt, vous pouvez afficher la correction sans changer de branche :

```
git show jour5-k6-performance-documents:k6/documents.js
git diff jour5-k6-performance-documents -- config/cache.php app/Http/Controllers/DocumentController.php .env.example
```

`git diff` ne montre pas un nouveau fichier non suivi : ouvrez aussi votre propre `k6/documents.js` dans VS Code. Pour changer de branche, sauvegardez d'abord votre travail et vérifiez `git status`. Un fichier `k6/documents.js` non suivi peut empêcher le changement : conservez-le avant de continuer, sans forcer Git.

Questions de compréhension

1. Quel est le rôle d'un VU ? Quelle différence y a-t-il entre VU et requêtes par seconde ?
2. Pourquoi notre scénario envoie-t-il un token, et où ce token est-il conservé ?
3. Que représentent `http_req_duration`, `http_reqs` et `http_req_failed` ?
4. Pourquoi faut-il distinguer les réponses 200 des réponses 429 dans une comparaison ?
5. Que se passe-t-il lors d'une lecture avec un cache vide, puis lors d'une lecture suivante ?
6. Pourquoi tous les VU de ce TD partagent-ils le même quota ?

Pour tester un autre serveur ou un VPS, utilisez uniquement une application pour laquelle vous avez l'autorisation et adaptez la charge à cet environnement.

Utiliser la correction

Le fichier `k6/documents.js` accepte des paramètres sans changer son code. Après avoir obtenu `$env:K6_TOKEN` comme ci-dessus, lancez :

```
docker run --rm --network formation-laravel_default -v "${PWD}:/scripts" -e K6_TOKEN -e VUS=2 -e DURATION=3s
grafana/k6:1.4.2 run /scripts/k6/documents.js
```

Modifiez `VUS` pour comparer 1, 2 et 5 utilisateurs. La mesure `lecture_200_ms` concerne uniquement les lectures réussies. Les compteurs `reponses_200`, `reponses_429` et `autres_reponses` apparaissent lorsqu'au moins une réponse de ce type a été reçue.

Pour envoyer exactement six requêtes après expiration du quota :

```
docker run --rm --network formation-laravel_default -v "${PWD}:/scripts" -e K6_TOKEN -e SCENARIO=limite grafana/k6:1.4.2
run /scripts/k6/documents.js
```

Le check `lecture réussie (200)` et la métrique HTTP signalent les 429 comme des échecs de lecture : cela permet d'observer la protection, même si ces réponses sont attendues pendant ce test. Pour tester un autre serveur autorisé, fournissez également `-e BASE_URL=https://adresse-du-serveur`.